Reviewer Pack — Finite Plane Line Count Bounds MCSP Complexity

Entry f2f7b7a70710 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 22:33:07 UTC

Finite Plane Line Count Bounds MCSP Complexity

Entry ID: `f2f7b7a70710` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 22:33:07 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry **Field B** (complexity object): Minimum Circuit Size (MCSP)

Statement:

> For a 3-SAT instance constructed from a finite projective plane of order q, the minimal circuit size required to solve MCSP is $\Theta(q^2)$. Specifically, if the instance encodes the incidence relations of lines and points in PG(2,q), then $MCSP(\Phi) \geq q^2/2$ and $MCSP(\Phi) \leq 2q^2$ for all $q \geq 2$.

Rationale (proposer's reasoning):

> Finite geometries impose combinatorial regularity on solution spaces, which may correlate with the inherent complexity of describing these structures via circuits. The projective plane's symmetry and incidence constraints could create unavoidable computational bottlenecks in MCSP.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1bd68af880cf3f34`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def finite_projective_plane(q):
    points = list(range(q * q + q + 1))
    lines = []
    for i in range(q * q + q + 1):
        line = set()
        for j in range(q * q + q + 1):
            if (i * j) % (q * q + q + 1) == 0:
                line.add(j)
        lines.append(line)
    return points, lines

def cnf_from_plane(points, lines):
    clauses = []
    for point in points:
        for line in lines:
            clause = {point: True}
            for p in line:
                if p != point:
                    clause[p] = False
            clauses.append(clause)
    return clauses

def solve(variables, assignment):
    for var, val in assignment.items():
        variables[var] = val
    for var, val in variables.items():
        if not all(assignment[var] == val for var, val in assignment.items()):
            return False
    return True

def branch_and_bound(cnf):
    variables = {var: None for var in cnf[0].keys()}
    def backtrack(assignment):
        unassigned = [var for var in variables if variables[var] is None]
        if not unassigned:
            return solve(variables, assignment)
        var = unassigned[0]
```

```
        for val in [True, False]:
            assignment[var] = val
            if backtrack(assignment):
                return True
            assignment[var] = None
        return False
    return backtrack({})

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q_values = [2, 3]
    results = []
    for q in q_values:
        points, lines = finite_projective_plane(q)
        clauses = cnf_from_plane(points, lines)
        mcsp_value = branch_and_bound(cnf=clauses)
        if not (q**2 / 2 <= mcsp_value <= 2 * q**2):
            return {
                "metric_name": "MCSP Complexity",
                "metric_value": mcsp_value,
                "instances_tested": len(clauses),
                "conjecture_holds": False,
                "counterexample": f"q={q}, MCSP(Φ)={mcsp_value} (not in [{q**2 / 2}, {2 * q**2}])"
            }
    return {
        "metric_name": "MCSP Complexity",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(clauses),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        if not result["conjecture_holds"]:
            break
    else:
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    if not result["conjecture_holds"]:
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
```

```
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 70, in run_trial
    mcsp_value = branch_and_bound(cnf=clauses)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 61, in branch_and_bo
    return backtrack({})
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 57, in backtrack
    if backtrack(assignment):
       ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 57, in backtrack
    if backtrack(assignment):
       ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76d345b.py", line 57, in backtrack
    if backtrack(assignment):
       ^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 994 more times]
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with recursion depth error, preventing evaluation of MCSP bounds. | next: Optimize branch-and-bound algorithm with memoization or increase recursion depth limits

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	64743
2	preregistration	ollama_remote	qwen3:8b	0	0	19995
3	novelty	ollama_remote	qwen3:8b	0	0	16546
4	novelty	ollama_remote	qwen3:8b	0	0	11130
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13506
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8879
7	judge	ollama_remote	qwen3:8b	0	0	14176

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 148974 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/f2f7b7a70710.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f2f7b7a70710.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f2f7b7a70710.tar.gz` (if generated)